TabTest: benchmark tabular e Large Data Model

Documentação de tudo que foi implementado no repositório

23 de outubro de 2026

Sumário

1 Visão geral 2
2 Dados: `download_datasets.py` 2
3 Problemas sugeridos: `src/tasks.py` 3
4 Pipeline comum 4
4.1 Carregamento e split: `src/data.py` 4
4.2 Pré-processamentos: `src/preprocessing.py` 4
4.3 Métricas: `src/metrics.py` 4
4.4 Interface dos modelos 4
5 Modelos 5
5.1 XGBoost: `models/xgb.py` 5
5.2 Regressão logística: `models/logistic.py` 5
5.3 Regressão probit: `models/probit.py` 5
5.4 Ridge: `models/ridge.py` 6
5.5 TabPFN: `models/tabpfn_model.py` 6
5.6 TabSTAR: `models/tabstar_model.py` 6
6 Resultados obtidos 7
7 Large Data Model tabular: `src/ldm/` 7
7.1 Ideia 7
7.2 Texto sem vocabulário fixo: `text.py` 8
7.3 Codificador por tabela: `table.py` 8
7.4 Embeddings das opções e da entrada: `model.py` 8
7.5 Head de autocomplete de máscaras 9
7.6 Objetivo de difusão: `diffusion.py` 9
7.7 Pré-treino em várias tabelas: `corpus.py`, `pretrain.py` 10
7.8 Adaptação a uma tabela nova: `predict.py`, `models/ldm_model.py` 10
7.9 Resultados do LDM 11
8 Ambiente e instalação 11
9 Limitações e próximos passos 12
10 Referências 12

1 Visão geral

O projeto tem três partes:

1. **Dados** (`download_datasets.py`): baixa datasets tabulares do scikit-learn e do OpenML e salva cada um como CSV em `data/`.
2. **Benchmark** (`src/`): define problemas de classificação e regressão sobre esses CSVs e compara XGBoost, regressão logística, regressão probit, Ridge, TabPFN e TabSTAR com um mesmo split e as mesmas métricas.
3. **Large Data Model (LDM) tabular** (`src/ldm/`): um único Transformer pré-treinado em muitas tabelas, que completa células mascaradas. Nenhum parâmetro depende de uma tabela específica, então ele se aplica a tabelas que nunca viu. O objetivo de treino é de difusão mascarada.

Aqui, **LDM** significa **Large Data Model**: um modelo fundacional para dados tabulares. Não tem relação com *Latent Diffusion Model*, porque não há autoencoder nem espaço latente. A difusão usada é discreta e acontece direto nas células.

```
TabTest/
  download_datasets.py     baixa os datasets -> data/*.csv
  requirements.txt
  src/
    tasks.py               15 problemas sugeridos
    data.py                carrega o CSV, codifica o alvo, faz o split
    preprocessing.py       one-hot/escala, category, ordinal
    metrics.py             métricas por tipo de tarefa
    run.py                 CLI do benchmark
    models/
      base.py              dataclass Prediction
      xgb.py  logistic.py  probit.py  ridge.py
      tabpfn_model.py  tabstar_model.py  ldm_model.py
    ldm/
      text.py              texto -> ids de hash (nomes de colunas e categorias)
      table.py             TableEncoder: bins, categorias, encode/decode, meta
      model.py             TabularLDM: Transformer independente da tabela + heads
      diffusion.py         MaskedDiffusion: loss, impute, generate, predict
      corpus.py            TableCorpus: lotes de várias tabelas
      corpus_download.py   34 tabelas do OpenML -> data/corpus/
      pretrain.py          CLI de pré-treino + checkpoints
      predict.py           adaptação a tabela nova + predição
      README.md
  tex/implementacao.typ    este documento
```

Notação. $X \in \mathbb{R}^{n \times p}$ é a matriz de features, com n linhas e p colunas. y é o vetor alvo e x_i é a i -ésima linha. K é o número de classes. No LDM, C é o número de colunas de uma tabela (alvo incluído) e K_c é o número de opções da coluna c.

2 Dados: `download_datasets.py`

Para cada dataset, o script chama o loader do scikit-learn com `as_frame=True` e salva o `frame` (features e alvo) em `data/<nome>.csv`.

Origem	Datasets	Observação
Embutidos (sem internet)	iris, wine, breast_cancer, diabetes, linnerud	vêm com o pacote
`fetch_*` / OpenML	california_housing, adult, titanic, house_prices, credit_g, bike_sharing, kc_house, covertype	cache em `~/scikit_learn`

- `--offline` baixa só os embutidos, `--list` lista os disponíveis, e também é possível passar nomes avulsos.
- Se um download falha, o script mostra o erro e segue para o próximo.
- **Correção aplicada:** o parser `pandas` de ARFF do scikit-learn quebra no titanic, no bike_sharing e no kc_house com `TypeError` em `strip_single_quotes`. A função `openml()` tenta o parser padrão e, se der `TypeError`, repete com `parser="liac-arff"`.

3 Problemas sugeridos: `src/tasks.py`

Cada `Task` (dataclass imutável) informa o CSV, a coluna alvo e o tipo (`binary`, `multiclass` ou `regression`). Também pode trazer colunas a descartar (`drop`), uma função que cria um alvo derivado (`derive`) e um limite de linhas (`max_rows`).

Tarefa	Tipo	Alvo	Comentário
adult_income	binária	`class`	renda > 50K; mistura num/cat, nulos
titanic_survival	binária	`survived`	descarta `boat` e `body`, que entregam a resposta, e também `name`, `ticket`, `home.dest`
credit_risk	binária	`class`	German Credit, quase tudo categórico
breast_cancer	binária	`target`	30 features numéricas, classes quase separáveis
california_expensive	binária	`expensive`	derivado: MedHouseVal > mediana
iris, wine	multiclasse	`target`	testes de sanidade, 3 classes
covertype	multiclasse	`Cover_Type`	7 classes, subamostrado para 20 000 linhas
california_price	regressão	`MedHouseVal`	preço mediano por bairro
house_prices	regressão	`SalePrice`	Ames, 79 features, muitos nulos
kc_house_price	regressão	`price`	King County
bike_demand	regressão	`count`	aluguéis por hora, efeitos de calendário/clima
diabetes	regressão	`target`	só 442 linhas, um regime de poucos dados
adult_hours	regressão	`hours-per-week`	alvo alternativo do mesmo dataset
titanic_fare	regressão	`fare`	preço da passagem

4 Pipeline comum

4.1 Carregamento e split: `src/data.py`

1. `load_frame` lê o CSV, aplica `derive` e remove as colunas de `drop`.
2. `load_task` remove as linhas com alvo nulo e subamostra se houver `max_rows`.
3. Na classificação, o alvo passa a texto e é codificado com `LabelEncoder` para inteiros $0, ..., K-1$. Os rótulos originais ficam em `Split.classes`, na mesma ordem.
4. O split treino/teste é 80/20 e estratificado na classificação. A semente padrão é 0.
5. `subsample(X, y, n)` limita o treino a n linhas sem reposição. TabPFN e TabSTAR usam essa função.

Uma coluna é categórica quando seu dtype não é numérico. Essa é a regra das propriedades `Split.cat_cols` e `Split.num_cols`.

4.2 Pré-processamentos: `src/preprocessing.py`

- `linear_preprocessor` (lineares):
  - numéricas: imputação pela mediana, depois padronização $z = (x - \mu)/\sigma$;
  - categóricas: imputação pela moda, depois one-hot. Categorias com menos de 10 ocorrências viram "infrequentes".
  - Com `drop_first=True`, usado no probit, a primeira dummy de cada variável é descartada para evitar a armadilha das dummies: com intercepto, as m dummies de uma variável somam 1 e a matriz fica singular.
- `to_category` (XGBoost): converte colunas de texto para `category`, usando as categorias vistas no treino. Categoria nova no teste vira NaN.
- `ordinal_encode` (TabPFN): devolve uma matriz numérica com as categóricas como inteiros, mantém os NaN e retorna os índices das colunas categóricas.

4.3 Métricas: `src/metrics.py`

- **Classificação:** acurácia, acurácia balanceada (média do recall por classe), log-loss $-\frac{1}{n}\sum_i \log \hat{p}_{i,y_i}$ e ROC-AUC (one-vs-rest no multiclasse).
- **Regressão:** RMSE $= \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$, MAE e $R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$.

4.4 Interface dos modelos

Cada módulo em `src/models/` expõe duas coisas:

- `KINDS`: os tipos de tarefa que o modelo suporta;
- `fit_predict(split, seed, **kw)`: devolve um `Prediction(y_pred, proba, model)`.

`run.py` importa cada módulo sob demanda, então um pacote ausente só faz aquele modelo ser pulado. Modelos incompatíveis com a tarefa também são pulados, e qualquer exceção vira "FALHOU" sem parar o benchmark.

```
python -m src.run --list
python -m src.run --task adult_income titanic_survival
python -m src.run --all --models xgboost logistic probit ridge --out results.csv
python -m src.run --task iris --models tabpfn --tabpfn-version V2
```

5 Modelos

5.1 XGBoost: `models/xgb.py`

Gradient boosting de árvores. O modelo é uma soma de M árvores, $\hat{y}_i = \sum_{m=1}^{M} f_m(x_i)$. Cada nova árvore f_m minimiza uma aproximação de segunda ordem da perda regularizada:

$$\mathcal{L}^{(m)} \approx \sum_i \Big[g_i f_m(x_i) + \frac{1}{2} h_i f_m(x_i)^2 \Big] + \gamma T + \frac{1}{2}\lambda \|w\|^2,$$

- g_i e h_i são o gradiente e a hessiana da perda em relação à predição atual;
- T é o número de folhas e w são os pesos das folhas.

Configuração usada:

- 500 árvores, `learning_rate` 0.05, profundidade 6, `subsample` e `colsample_bytree` 0.8, `tree_method="hist"`;
- `enable_categorical=True`: as categóricas são divididas nativamente, sem one-hot;
- os NaN seguem uma direção padrão aprendida em cada split.

Não há pesos pré-treinados: o modelo é treinado do zero.

5.2 Regressão logística: `models/logistic.py`

No caso binário, $P(y = 1 \mid x) = \sigma(\beta_0 + x^\top \beta)$, com $\sigma(z) = 1/(1 + e^{-z})$. No multiclasse, usa softmax:

$$P(y = k \mid x) = \frac{\exp(\beta_{0k} + x^\top \beta_k)}{\sum_{j=1}^{K} \exp(\beta_{0j} + x^\top \beta_j)}.$$

O scikit-learn minimiza a log-verossimilhança negativa mais uma penalidade $\frac{1}{2C}\|\beta\|^2$, com $C = 1$. Por isso a logística segue estável mesmo quando as classes são separáveis.

5.3 Regressão probit: `models/probit.py`

A ligação passa a ser a CDF da normal padrão, Φ:

$$P(y = 1 \mid x) = \Phi(x^\top \beta), \quad \Phi(z) = \int_{-\infty}^{z} \frac{1}{\sqrt{2\pi}} e^{-u^2/2}\, \mathrm{d}u.$$

Uma forma equivalente usa uma variável latente: $y^* = x^\top \beta + \varepsilon$, com $\varepsilon \sim \mathcal{N}(0, 1)$ e $y = \mathbb{1}[y^* > 0]$. O estimador é o de máxima verossimilhança:

$$\ell(\beta) = \sum_{i=1}^{n} [y_i \log \Phi(x_i^\top \beta) + (1 - y_i) \log(1 - \Phi(x_i^\top \beta))].$$

Detalhes da implementação (`ProbitClassifier`, compatível com o scikit-learn):

- usa `statsmodels.Probit` com intercepto (`add_constant`), ajustado por **L-BFGS**. O Newton padrão falha com Hessiana singular quando há colunas colineares, como `education` e `education-num` no adult;
- o probit é binário por natureza. No multiclasse, ajusto um modelo one-vs-rest por classe, $s_k = \Phi(x^\top \beta_k)$, e renormalizo: $\hat{p}_k = s_k / \sum_j s_j$;
- `pred.model[-1].results_[k].summary()` dá coeficientes, erros-padrão e p-valores. É a vantagem inferencial do probit sobre os modelos de caixa-preta;
- não há regularização. Quando as classes são (quase) separáveis, como no breast_cancer, $\|\beta\| \to \infty$ e as probabilidades vão a 0 ou 1, o que explode o log-loss.

A relação entre os dois modelos: $\Phi(z) \approx \sigma(1.6z)$. Por isso as acurácias de logística e probit costumam ficar próximas, com coeficientes cerca de 1.6 vezes menores no probit.

5.4 Ridge: `models/ridge.py`

Logística e probit não servem para alvos contínuos, então a regressão tem um baseline linear próprio:

$$\hat{\beta} = \arg\min_{\beta} \|y - X\beta\|^2 + \alpha\|\beta\|^2 = (X^\top X + \alpha I)^{-1} X^\top y,$$

com $\alpha \in \{10^{-3}, ..., 10^3\}$ escolhido por validação cruzada (`RidgeCV`). Usa o mesmo pré-processamento linear.

5.5 TabPFN: `models/tabpfn_model.py`

O TabPFN é um Transformer pré-treinado em milhões de datasets sintéticos amostrados de um *prior* sobre processos geradores (modelos causais estruturais). Ele aprende a aproximar a distribuição preditiva posterior:

$$p(y_* \mid x_*, \mathcal{D}_{\text{treino}}) = \int p(y_* \mid x_*, \varphi)\, p(\varphi \mid \mathcal{D}_{\text{treino}})\, \mathrm{d}\varphi,$$

em que φ indexa o processo gerador dos dados. O treino entra como **contexto** da atenção, então `fit` não altera nenhum peso: é *in-context learning* em um único forward.

- **Pesos:** ficam no Hugging Face, na org `Prior-Labs` (`tabpfn_3`, `tabpfn_3_5`, `tabpfn_2_5`, …), e são baixados no primeiro uso.
  - Da versão 2.5 em diante são *gated* e não comerciais: é preciso aceitar a licença na página e fazer `hf auth login` (ou definir `HF_TOKEN`).
  - A v2 é Apache 2.0 com atribuição e pode ser escolhida com `--tabpfn-version V2`.
- **Entrada:** matriz `ordinal_encode` com NaN preservado e `categorical_features_indices`. Também passo `ignore_pretraining_limits=True`.
- **Limites:** o custo da atenção cresce com o tamanho do contexto, por isso o treino fica limitado a 10 000 linhas (`MAX_TRAIN`). Em CPU, acima de cerca de 1000 linhas, é preciso definir `TABPFN_ALLOW_CPU_LARGE_DATASET=1`.

5.6 TabSTAR: `models/tabstar_model.py`

Modelo fundacional tabular (Arazi, Shapira e Reichart, NeurIPS 2025) feito para tabelas com texto.

- **Como funciona:** cada célula é *verbalizada*, ou seja, nome da coluna e valor viram texto. Um encoder de texto pré-treinado gera as representações, e os rótulos do alvo também entram como texto ("target-aware"). A arquitetura não tem parâmetros específicos de dataset.
- **Pesos:** `alana89/TabSTAR`, baixados automaticamente no `fit()`.
- **Treino:** ao contrário do TabPFN, o `fit` faz **fine-tuning LoRA**: até 50 épocas, paciência 5, $r = 32$, `lr` 10^{-3}. Por isso a GPU é praticamente obrigatória, e o treino fica limitado a 5 000 linhas (`MAX_TRAIN`).
- **Entrada:** um `DataFrame` bruto e uma `Series`. Passo os rótulos originais (`split.classes[y]`), não os inteiros, porque o modelo usa o significado do texto.
- **Probabilidades:** assumo que as colunas de `predict_proba` seguem a ordem ordenada dos rótulos. Se o formato não bater, só as predições duras são avaliadas.
- **Ambiente:** requer Python 3.10–3.12.

6 Resultados obtidos

Execução real em CPU, com Python 3.12 e semente 0. TabPFN e TabSTAR ainda não rodaram porque `torch`, `tabpfn` e `tabstar` não estavam instalados.

Tarefa	**Métrica**	**XGBoost**	**Logística**	**Probit**	**Ridge**
iris	acurácia	0.933	0.967	0.967	–
breast_cancer	ROC-AUC	0.986	0.996	0.948	–
titanic_survival	ROC-AUC	0.864	0.863	0.860	–
adult_income	ROC-AUC	0.927	0.904	0.904	–
diabetes	R^2	0.236	–	–	0.339
california_price	R^2	0.854	–	–	0.594
titanic_fare	R^2	0.467	–	–	0.303

Leitura dos resultados:

- O XGBoost ganha onde há não-linearidade e interações (california, adult).
- Os lineares empatam ou vencem com poucos dados e sinal aproximadamente linear (diabetes, iris, breast_cancer).
- No breast_cancer, o probit teve log-loss de 1.90, contra 0.08 da logística. É o efeito da separação sem regularização descrito acima.

7 Large Data Model tabular: `src/ldm/`

7.1 Ideia

Um **Large Data Model** é o análogo tabular de um modelo de linguagem fundacional: **um único modelo, pré-treinado em muitas tabelas**, que depois se aplica a uma tabela nova com pouco ou nenhum treino. O desafio é que cada tabela tem colunas diferentes, em número, nome, tipo e escala. Por isso o requisito central da arquitetura é:

nenhum parâmetro pode depender de uma tabela específica.

Na primeira versão do backbone, as colunas e os valores eram ids aprendidos por tabela (um vocabulário com *offset* por coluna e uma head por coluna). Um modelo assim só funciona na tabela em que foi treinado. A versão atual troca todos esses ids por **descrições** que fazem sentido em qualquer tabela:

Objeto	**Antes (específico da tabela)**	**Agora (independente da tabela)**
coluna c	$E_{\text{coluna}}[c]$, um id	embedding do **texto do nome** da coluna
categoria	id com *offset* por coluna	embedding do **texto da categoria**
bin numérico b	id com *offset* por coluna	posição do bin no CDF (q) e mediana padronizada (z)
head de saída	uma matriz W_c por coluna	**produto interno** com os embeddings das opções

Cada linha é uma sequência de C células, e o modelo aprende a completar células mascaradas com uma taxa de máscara que varia de 0 a 1 (difusão discreta com estado absorvente, MDLM). O mesmo modelo serve para:

- **predição:** mascarar só o alvo dá p(alvo | resto);

- **imputação:** mascarar as células faltantes;
- **geração sintética:** começar com tudo mascarado.

7.2 Texto sem vocabulário fixo: `text.py`

Nomes de colunas e categorias viram um saco de **palavras** e **trigramas de caracteres**. O camelCase e o snake_case são separados antes:

$$\text{MedHouseVal} \to \{\text{w:med}, \text{w:house}, \text{w:val}, \text{c:<me}, \text{c:med}, \text{c:ed>}, ...\}$$

Cada pedaço é mapeado por hash (`crc32`, estável entre processos) para um de $B = 2^{16}$ ids, e o id 0 fica reservado para padding. O embedding de um texto s com pedaços $u_1, ..., u_m$ é

$$\text{TextEmb}(s) = W\left(\text{LN}\left(\frac{1}{m}\sum_{i=1}^{m} E_{\text{hash}}[h(u_i)]\right)\right) + b.$$

Um nome nunca visto cai no mesmo espaço: "hours-per-week" compartilha pedaços com "hours", e "MedHouseVal" com "house_value". Trigramas dão robustez a abreviações e variações de grafia.

7.3 Codificador por tabela: `table.py`

O `TableEncoder` é ajustado em cada tabela. Ele só calcula estatísticas, sem nenhum parâmetro treinável.

- **Tipo:** numérica com mais de 10 valores distintos é `NUM`; o resto, incluindo texto e booleanos, é `CAT`.
- **Numérica:** até 32 bins por quantis do treino ($q_1 < ... < q_{K_c-1}$, colapsando quantis repetidos). Cada bin b é descrito por:
  - sua posição no CDF, $q_b = (N_{<b} + N_b/2)/N$, onde $N_{<b}$ é o número de valores nos bins anteriores e N_b o número no próprio bin;
  - sua mediana padronizada, $z_b = (\text{mediana}_b - \mu)/\sigma$, cortada em $[-5, 5]$.

  A célula guarda o índice do bin e o valor contínuo $z = (x - \mu)/\sigma$.
- **Categórica:** mantém as 63 categorias mais frequentes com pelo menos 2 ocorrências. As demais viram `<other>`. A coluna alvo de uma classificação nunca é cortada (`exact`).
- **Índice local:** $0 =$ `[NULL]` (valor faltante) e $1, ..., K_c =$ opção da coluna.
- `meta()` devolve a descrição da tabela que o modelo consome: tipo, número de opções, ids do nome, ids das categorias, q e z de cada bin.

7.4 Embeddings das opções e da entrada: `model.py`

Para cada coluna c e opção k, o embedding da opção é

$$o_{ck} = \begin{cases} \text{TextEmb}(\text{categoria}_k) & \text{se } c \text{ é categórica} \\ \text{MLP}(\varphi(q_{ck}) \oplus \varphi(z_{ck}/5)) & \text{se } c \text{ é numérica} \end{cases}, \quad o_{c0} = e_{[\text{NULL}]},$$

com features de Fourier $\varphi(x) = \left[x, \sin(2^j\pi x), \cos(2^j\pi x)\right]_{j<8}$ e $\oplus$ a concatenação. Como q é a posição no CDF, "o bin mais alto" tem o mesmo significado em qualquer coluna numérica de qualquer tabela.

A entrada da célula (n, c), com índice local l_{nc}, é

$$h_{nc}^{(0)} = v_{nc} + \underbrace{\text{TextEmb}(\text{nome}_c) + E_{\text{tipo}}[c]}_{\text{coluna}} + \text{MLP}(\text{senoidal}(r_n)),$$

$$v_{nc} = \begin{cases} e_{[\text{MASK}]} & \text{se a célula está mascarada} \\ W_{\text{in}} o_{c,l_{nc}} + \mathbb{1}_{\text{num. visível}} \cdot \text{MLP}(\varphi(z_{nc}/5)) & \text{caso contrário} \end{cases}$$

- r_n é a fração de células mascaradas da linha n;
- o termo $\text{MLP}(\varphi(z))$ injeta o valor numérico exato, além do bin.

Não há embedding posicional: a atenção é equivariante a permutações das colunas, e as colunas de uma tabela não têm ordem.

Encoder. L blocos **pre-LN** bidirecionais, sem máscara causal: $h \leftarrow h + \text{MHA}(\text{LN}(h))$, seguido de $h \leftarrow h + \text{MLP}(\text{LN}(h))$. Os padrões do pré-treino são $d = 256$, $L = 6$ e 8 cabeças; nos testes em CPU usei $d = 128$, $L = 4$, cerca de 9 M de parâmetros, dos quais 8.4 M estão na tabela de hash.

7.5 Head de autocomplete de máscaras

A head não tem pesos por coluna. Ela pontua o estado da célula contra os embeddings das opções válidas da coluna:

$$\text{logit}_{nck} = \frac{\langle W_q h_{nc}^{(L)}, W_{\text{out}} o_{ck} \rangle}{\sqrt{d}}, \quad k \in \{0, ..., K_c\},$$

- opções inexistentes ($k > K_c$) recebem $-\infty$;
- a head se aplica a qualquer coluna de qualquer tabela, inclusive com categorias nunca vistas no pré-treino, desde que o texto seja informativo;
- o mesmo o_{ck} entra na entrada (via W_{in}) e na saída (via W_{out}), um *weight tying* análogo ao dos modelos de linguagem.

A **head de regressão**, $\hat{z}_{nc} = w^\top h_{nc}^{(L)} + b$, prevê o valor contínuo padronizado das numéricas.

7.6 Objetivo de difusão: `diffusion.py`

Forward. Com o schedule $\alpha(t)$ (probabilidade de a célula continuar visível),

$$\alpha_{\cos}(t) = \cos(\pi t/2) \quad \text{(padrão)}, \quad \alpha_{\text{linear}}(t) = 1 - t,$$

cada célula é mascarada de forma independente:

$$q(x_t^c \mid x_0^c) = \alpha(t)\, \delta_{x_0^c} + (1 - \alpha(t))\, \delta_{[\text{MASK}]}.$$

Loss. Sorteio $t \sim \mathcal{U}(10^{-3}, 1)$ e uso a cota contínua do ELBO do MDLM, mais o MSE da head de regressão:

$$\mathcal{L}(\theta) = \mathbb{E}\left[w(t) \cdot \frac{1}{C} \sum_{c \in \mathcal{M}} -\log p_\theta(x_0^c \mid x_t) \right] + \lambda\, \mathbb{E}\left[\frac{1}{|\mathcal{M}_{\text{num}}|} \sum_{c \in \mathcal{M}_{\text{num}}} (\hat{z}_c - z_c)^2 \right],$$

- $\mathcal{M}$ é o conjunto das células mascaradas e $\mathcal{M}_{\text{num}}$ o das numéricas mascaradas não nulas;
- $w(t) = -\alpha'(t)/(1 - \alpha(t))$, que vale $1/t$ no schedule linear e $(\pi/2)\sin(\pi t/2)/(1 - \cos(\pi t/2))$ no cosseno, limitado a 10^3;
- $\lambda = 1$;
- a cross-entropy é sobre as $K_c + 1$ opções da coluna, incluindo `[NULL]`. Assim o modelo também aprende **quando** um valor costuma faltar.

Condicionamento. O modelo recebe a fração de máscara **observada** r, e não o t sorteado. Na imputação, parte da linha é conhecida e t perde o sentido, mas r continua bem definido.

Reverse. As células desconhecidas começam mascaradas. Para $t = 1, (S-1)/S, ..., 1/S$ e $s = t - 1/S$:

1. calcula $p_\theta(x_0^c \mid x_t)$;
2. amostra um candidato (com temperatura) ou toma o argmax;
3. revela cada célula mascarada com probabilidade

$$P(\text{revelar}) = \frac{\alpha(s) - \alpha(t)}{1 - \alpha(t)},$$

que é o posterior exato do processo absorvente (estratégia `random`). A alternativa `confidence` revela as mais confiantes.

Numéricas reveladas recebem como z a mediana padronizada do bin escolhido. Células reveladas não voltam a ser mascaradas.

Predição. `predict` mascara só as colunas alvo e roda um forward, com `[NULL]` proibido:

- **classificação:** a probabilidade de cada classe é o softmax sobre as opções do alvo;
- **regressão:** $\hat{y} = \hat{z}\sigma + \mu$, a head de regressão na escala original. A alternativa é o valor esperado dos bins, $\sum_b p_b \text{mediana}_b$ (`expected_bin_value`).

7.7 Pré-treino em várias tabelas: `corpus.py`, `pretrain.py`

- **Corpus:** `corpus_download.py` baixa 34 tabelas do OpenML para `data/corpus/`, com prefixo `openml_`. Entre elas: phoneme, spambase, bank-marketing, electricity, MagicTelescope, mushroom, letter, wine_quality, pol, elevators, nursery e outras.
  - Tabelas de avaliação ficam de fora.
  - O `houses` do OpenML foi removido, porque é o California Housing e vazaria as tarefas `california_*`.
- **Amostragem:** cada passo sorteia uma tabela com probabilidade $\propto n_{\text{linhas}}^{0.5}$, o que dá mais peso relativo às pequenas. Depois sorteia um lote de linhas e um subconjunto de até 64 colunas.
  - Um lote tem uma tabela só, então todas as linhas compartilham o mesmo `meta`.
  - Cortar colunas também é aumento de dados: o modelo aprende a prever a partir de contextos parciais.
- **Otimização:** AdamW, warmup linear seguido de cosseno, *clipping* em 1.0.
- **Checkpoint:** guarda a configuração, os pesos, o schedule e a **lista das tabelas vistas**. O `ldm_model` usa essa lista para avisar sobre vazamento.

7.8 Adaptação a uma tabela nova: `predict.py`, `models/ldm_model.py`

1. Ajusta o `TableEncoder` no treino da tabela nova, forçando o tipo do alvo (`CAT` na classificação, `NUM` na regressão).
2. **Zero-shot** (`--ldm-steps 0`): prediz direto com o modelo pré-treinado.
3. **Fine-tuning** (padrão de 300 passos, `lr` 10^{-4}): uma cópia do modelo treina na tabela nova, alternando duas losses:
   - com probabilidade 0.5, a loss de difusão comum, que preserva o modelo da tabela inteira;
   - no resto, lotes em que o alvo é sempre mascarado (a loss da tarefa, com média uniforme).
4. **Sem checkpoint:** o mesmo procedimento treina do zero (2000 passos, `lr` 10^{-3}). É a ablação que mede o ganho do pré-treino.

```
python -m src.ldm.corpus_download
python -m src.ldm.pretrain --dirs data/corpus --steps 20000 --save checkpoints/
ldm.pt
python -m src.run --task adult_income --models ldm --ldm-checkpoint checkpoints/
ldm.pt --ldm-steps 0
python -m src.run --task adult_income --models ldm xgboost --ldm-checkpoint
```

```
checkpoints/ldm.pt
python -m src.run --task adult_income --models ldm       # do zero
```

7.9 Resultados do LDM

Configuração:

- **Pré-treino:** checkpoint `checkpoints/ldm_small.pt`. Modelo com $d = 128$, $L = 4$ e 9.3 M de parâmetros, treinado em CPU por 3000 passos (lote 128, cerca de 10 min) nas 34 tabelas de `data/corpus` (256 mil linhas). A cross-entropy nas células mascaradas caiu de 2.44 para cerca de 2.05.
- **Sem vazamento:** nenhuma tabela de avaliação está no corpus.
- **Comparação justa:** fine-tuning e treino do zero usam o **mesmo número de passos** (300) e a mesma arquitetura, então a diferença entre eles mede só o efeito do pré-treino.

Tarefa	Métrica	LDM zero-shot	LDM pré-tr.+FT	LDM do zero	XGBoost	Linear
titanic_survival	ROC-AUC	0.429	**0.825**	0.737	0.864	0.863
credit_risk	ROC-AUC	0.532	**0.654**	0.464	0.811	0.830
breast_cancer	ROC-AUC	0.089	0.987	**0.990**	0.986	0.996
diabetes	R^2	0.094	0.244	**0.325**	0.236	0.339
california_price	R^2	0.054	**0.540**	0.052	0.854	0.594

"Linear" é a logística na classificação e o Ridge na regressão. Em negrito, o melhor entre as três versões do LDM.

Leitura dos resultados:

- **O pré-treino ajuda.** Com o mesmo orçamento de 300 passos, o modelo pré-treinado supera o treinado do zero em 3 de 5 tarefas, com destaque para o california (R^2 0.54 contra 0.05) e o titanic (AUC 0.83 contra 0.74). O modelo do zero ainda nem saiu do lugar no california, enquanto o pré-treinado já chega perto do Ridge.
  - No diabetes (442 linhas, 10 features numéricas), o modelo do zero foi melhor. Os bins e o espaço numérico aprendidos no corpus não se transferiram bem para essa tabela.
- **O zero-shot ainda não funciona.** Fica no nível do acaso, e no breast_cancer o AUC de 0.089 está **invertido**. É o esperado neste estágio:
  - o alvo é o texto "0"/"1", sem semântica;
  - os embeddings de hash não têm significado prévio;
  - 34 tabelas não bastam para o modelo aprender relações gerais.

  O zero-shot real exige um corpus muito maior, um encoder de texto semântico e, de preferência, linhas de treino como contexto (TabPFN).
- **O LDM ainda perde para os modelos clássicos.** Ele fica abaixo do XGBoost e dos lineares, exceto no breast_cancer e no diabetes, onde empata. O pré-treino foi pequeno (10 min de CPU, $d = 128$) e o fine-tuning curto, sem ajuste de hiperparâmetros. O resultado valida o pipeline, mas ainda não demonstra um modelo competitivo. Os próximos passos estão na seção de limitações.

8 Ambiente e instalação

O Python do sistema (3.14) não tem `python3-venv`, e o TabSTAR exige Python 3.10–3.12. O caminho recomendado é o `uv`:

```
uv venv -p 3.12 .venv312
uv pip install -p .venv312/bin/python -r requirements.txt
.venv312/bin/python download_datasets.py
.venv312/bin/python -m src.ldm.corpus_download
.venv312/bin/python -m src.run --all --out results.csv
```

Para o LDM, o pré-treino deve rodar em GPU. Em CPU, cada passo atualiza densamente toda a tabela de hash ($2^{16} \times d$) no AdamW e fica lento.

9 Limitações e próximos passos

- **Benchmark:**
  - não há busca de hiperparâmetros nem early stopping no XGBoost;
  - avaliação com um único split: o ideal é repetir com várias sementes ou usar CV e reportar média ± desvio;
  - o probit não tem regularização, então em dados separáveis o ideal é penalizar ou usar Firth.
- **TabPFN e TabSTAR:** ainda não foram executados. Faltam GPU, `torch` e o aceite da licença dos pesos do TabPFN v2.5+.
- **LDM:**
  - **corpus pequeno:** 34 tabelas é pouco para um modelo fundacional. O próximo passo é escalar para milhares de tabelas (OpenML, Kaggle) e/ou tabelas sintéticas geradas por modelos causais estruturais, como faz o TabPFN;
  - **texto:** o embedding é um saco de n-gramas com hash, sem semântica real. Um encoder de frases pré-treinado faria "salary" ficar perto de "income", como no CARTE e no TabSTAR;
  - **contexto entre linhas:** cada linha é processada isoladamente. Atender também a outras linhas da tabela (atenção entre linhas, no estilo TabPFN) daria aprendizado em contexto;
  - não há validação nem early stopping no fine-tuning, e a qualidade das amostras sintéticas não foi avaliada;
  - colunas de texto livre de alta cardinalidade, como `name` no titanic, viram quase tudo `<other>`.

10 Referências

- Chen & Guestrin (2016). *XGBoost: A Scalable Tree Boosting System*. KDD.
- Hollmann et al. (2025). *Accurate predictions on small data with a tabular foundation model*. Nature. Pesos em huggingface.co/Prior-Labs.
- Arazi, Shapira & Reichart (2025). *TabSTAR: A Tabular Foundation Model for Tabular Data with Text Fields*. NeurIPS. github.com/alanarazi7/TabSTAR.
- Sahoo et al. (2024). *Simple and Effective Masked Diffusion Language Models* (MDLM). NeurIPS.
- Austin et al. (2021). *Structured Denoising Diffusion Models in Discrete State-Spaces* (D3PM). NeurIPS.
- Kim, Grinsztajn & Varoquaux (2024). *CARTE: Pretraining and Transfer for Tabular Learning*. ICML. (Colunas e valores descritos por texto para transferência entre tabelas.)
- Zhu et al. (2023). *XTab: Cross-table Pretraining for Tabular Transformers*. ICML.